

入口点，也就是 `_start` 函数的地址。这个函数是在系统目标文件 `ctrl.o` 中定义的，对所有的 C 程序都是一样的。`_start` 函数调用系统启动函数 `__libc_start_main`，该函数定义在 `libc.so` 中。它初始化执行环境，调用用户层的 `main` 函数，处理 `main` 函数的返回值，并且在需要的时候把控制返回给内核。

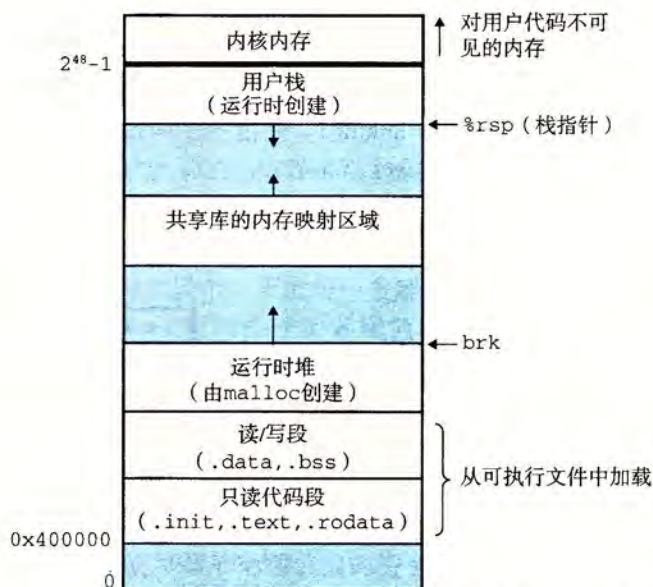


图 7-15 Linux x86-64 运行时内存映像。没有展示出由于段对齐要求和地址空间布局随机化 (ASLR) 造成的空隙。区域大小不成比例

旁注 加载器实际是如何工作的？

我们对于加载的描述从概念上来说是正确的，但也不是完全准确，这是有意为之。要理解加载实际是如何工作的，你必须理解进程、虚拟内存和内存映射的概念，这些我们还没有加以讨论。在后面第 8 章和第 9 章中遇到这些概念时，我们将重新回到加载的问题上，并逐渐向你揭开它的神秘面纱。

对于不够有耐心的读者，下面是关于加载实际是如何工作的一个概述：Linux 系统中的每个程序都运行在一个进程上下文中，有自己的虚拟地址空间。当 shell 运行一个程序时，父 shell 进程生成一个子进程，它是父进程的一个复制。子进程通过 `execve` 系统调用启动加载器。加载器删除子进程现有的虚拟内存段，并创建一组新的代码、数据、堆和栈段。新的栈和堆段被初始化为零。通过将虚拟地址空间中的页映射到可执行文件的页大小的片 (chunk)，新的代码和数据段被初始化为可执行文件的内容。最后，加载器跳转到 `_start` 地址，它最终会调用应用程序的 `main` 函数。除了一些头部信息，在加载过程中没有任何从磁盘到内存的数据复制。直到 CPU 引用一个被映射的虚拟页时才会进行复制，此时，操作系统利用它的页面调度机制自动将页面从磁盘传送到内存。

7.10 动态链接共享库

我们在 7.6.2 节中研究的静态库解决了许多关于如何让大量相关函数对应用程序可用的问题。然而，静态库仍然有一些明显的缺点。静态库和所有的软件一样，需要定期维护和更新。如果应用程序员想要使用一个库的最新版本，他们必须以某种方式了解到该库的

更新情况，然后显式地将他们的程序与更新了的库重新链接。

另一个问题是几乎每个 C 程序都使用标准 I/O 函数，比如 `printf` 和 `scanf`。在运行时，这些函数的代码会被复制到每个运行进程的文本段中。在一个运行上百个进程的典型系统上，这将对稀缺的内存系统资源的极大浪费。（内存的一个有趣属性就是不论系统的内存有多大，它总是一种稀缺资源。磁盘空间和厨房的垃圾桶同样有这种属性。）

共享库(shared library)是致力于解决静态库缺陷的一个现代创新产物。共享库是一个目标模块，在运行或加载时，可以加载到任意的内存地址，并和一个在内存中的程序链接起来。这个过程称为动态链接(dynamic linking)，是由一个叫做动态链接器(dynamic linker)的程序来执行的。共享库也称为共享目标(shared object)，在 Linux 系统中通常用 `.so` 后缀来表示。微软的操作系统大量地使用了共享库，它们称为 DLL(动态链接库)。

共享库是以两种不同的方式来“共享”的。首先，在任何给定的文件系统中，对于一个库只有一个 `.so` 文件。所有引用该库的可执行目标文件共享这个 `.so` 文件中的代码和数据，而不是像静态库的内容那样被复制和嵌入到引用它们的可执行的文件中。其次，在内存中，一个共享库的 `.text` 节的一个副本可以被不同的正在运行的进程共享。在第 9 章我们学习虚拟内存时将更加详细地讨论这个问题。

图 7-16 概括了图 7-7 中示例程序的动态链接过程。为了构造图 7-6 中示例向量例程的共享库 `libvector.so`，我们调用编译器驱动程序，给编译器和链接器如下特殊指令：

```
linux> gcc -shared -fpic -o libvector.so addvec.c multvec.c
```

`-fpic` 选项指示编译器生成与位置无关的代码(下一节将详细讨论这个问题)。`-shared` 选项指示链接器创建一个共享的目标文件。一旦创建了这个库，随后就要将它链接到图 7-7 的示例程序中：

```
linux> gcc -o prog21 main2.c ./libvector.so
```

这样就创建了一个可执行目标文件 `prog21`，而此文件的形式使得它在运行时可以和 `libvector.so` 链接。基本的思路是当创建可执行文件时，静态执行一些链接，然后在程序加载时，动态完成链接过程。认识到这一点是很重要的：此时，没有任何 `libvector.so` 的代码和数据节真的被复制到可执行文件 `prog21` 中。反之，链接器复制了一些重定位和符号表信息，它们使得运行时可以解析对 `libvector.so` 中代码和数据的引用。

当加载器加载和运行可执行文件 `prog21` 时，它利用 7.9 节中讨论过的技术，加载部分链接的可执行文件 `prog21`。接着，它注意到 `prog21` 包含一个 `.interp` 节，这一节包含动态链接器的路径名，动态链接器本身就是一个共享目标(如在 Linux 系统上的 `ld-linux.so`)。加载器不会像它通常所做地那样将控制传递给应用，而是加载和运行这个动态链接器。然

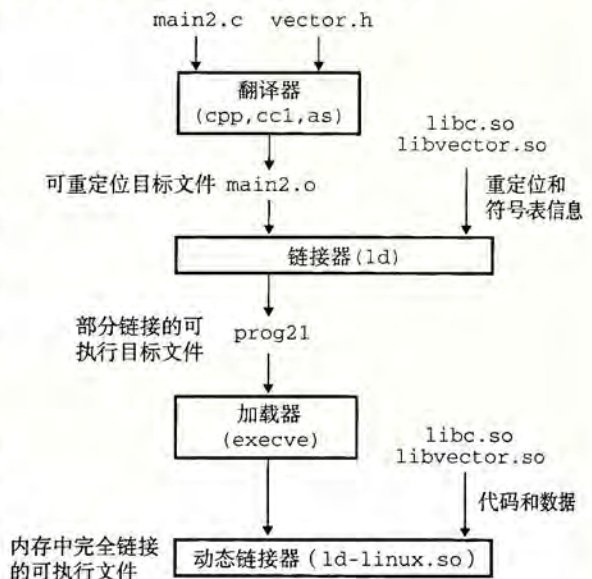


图 7-16 动态链接共享库

后，动态链接器通过执行下面的重定位完成链接任务：

- 重定位 `libc.so` 的文本和数据到某个内存段。
- 重定位 `libvector.so` 的文本和数据到另一个内存段。
- 重定位 `prog21` 中所有对由 `libc.so` 和 `libvector.so` 定义的符号的引用。

最后，动态链接器将控制传递给应用程序。从这个时刻开始，共享库的位置就固定了，并且在程序执行的过程中都不会改变。

7.11 从应用程序中加载和链接共享库

到目前为止，我们已经讨论了在应用程序被加载后执行前时，动态链接器加载和链接共享库的情景。然而，应用程序还可能在它运行时要求动态链接器加载和链接某个共享库，而无需在编译时将那些库链接到应用中。

动态链接是一项强大有用的技术。下面是一些现实世界中的例子：

- 分发软件。微软 Windows 应用的开发者常常利用共享库来分发软件更新。他们生成一个共享库的新版本，然后用户可以下载，并用它替代当前的版本。下一次他们运行应用程序时，应用将自动链接和加载新的共享库。
- 构建高性能 Web 服务器。许多 Web 服务器生成动态内容，比如个性化的 Web 页面、账户余额和广告标语。早期的 Web 服务器通过使用 `fork` 和 `execve` 创建一个子进程，并在该子进程的上下文中运行 CGI 程序来生成动态内容。然而，现代高性能的 Web 服务器可以使用基于动态链接的更有效和完善的方法来生成动态内容。

其思路是将每个生成动态内容的函数打包在共享库中。当一个来自 Web 浏览器的请求到达时，服务器动态地加载和链接适当的函数，然后直接调用它，而不是使用 `fork` 和 `execve` 在子进程的上下文中运行函数。函数会一直缓存在服务器的地址空间中，所以只要一个简单的函数调用的开销就可以处理随后的请求了。这对一个繁忙的网站来说是有很大影响的。更进一步地说，在运行时无需停止服务器，就可以更新已存在的函数，以及添加新的函数。

Linux 系统为动态链接器提供了一个简单的接口，允许应用程序在运行时加载和链接共享库。

```
#include <dlfcn.h>
```

```
void *dlopen(const char *filename, int flag);
```

返回：若成功则为指向句柄的指针，若出错则为 NULL。

`dlopen` 函数加载和链接共享库 `filename`。用已用带 `RTLD_GLOBAL` 选项打开了的库解析 `filename` 中的外部符号。如果当前可执行文件是带 `-rdynamic` 选项编译的，那么对符号解析而言，它的全局符号也是可用的。`flag` 参数必须要么包括 `RTLD_NOW`，该标志告诉链接器立即解析对外部符号的引用，要么包括 `RTLD_LAZY` 标志，该标志指示链接器推迟符号解析直到执行来自库中的代码。这两个值中的任意一个都可以和 `RTLD_GLOBAL` 标志取或。

```
#include <dlfcn.h>
```

```
void *dlsym(void *handle, char *symbol);
```

返回：若成功则为指向符号的指针，若出错则为 NULL。